

A SEEDSCALE HOW-TO GUIDE

How to Build a Branded Copywriting Slackbot Your Whole Team Can Use

A step-by-step guide to deploying on-brand, AI-powered copy your whole team can use.

BY SEEDSCALE AGENCY

SEEDSCALE AGENCY

Digital Consulting for Mission-Driven Organizations

How to Build a Branded Copywriting Slackbot Your Whole Team Can Use

Your marketing and comms team is the bottleneck for copy. Every email, every social post, every landing page runs through the same two or three people who know the brand voice. When they are out sick, on deadline, or drowning in requests, the whole company slows down.

This guide shows you how to build a Slackbot that any staff member can message, that writes in your exact brand voice, and that runs for around \$5 a month in hosting plus Claude API usage. An entry-level marketer can manage it. No developers required.

You will build it once. Your team will use it every day.

What This Solves

A branded copywriting Slackbot does three things:

- Lets every staff member get on-brand copy in under a minute, without pinging the marketing team.
- Enforces your voice, anti-AI rules, and platform best practices on every single piece of copy.
- Centralizes requests in Slack, where your team already works.

When a branded Slackbot handles the first draft, a 30-minute task for your comms team becomes a 2-minute task for the requester. Copy quality goes up, not down, because the bot runs the brand voice checklist every single time.

What You Need

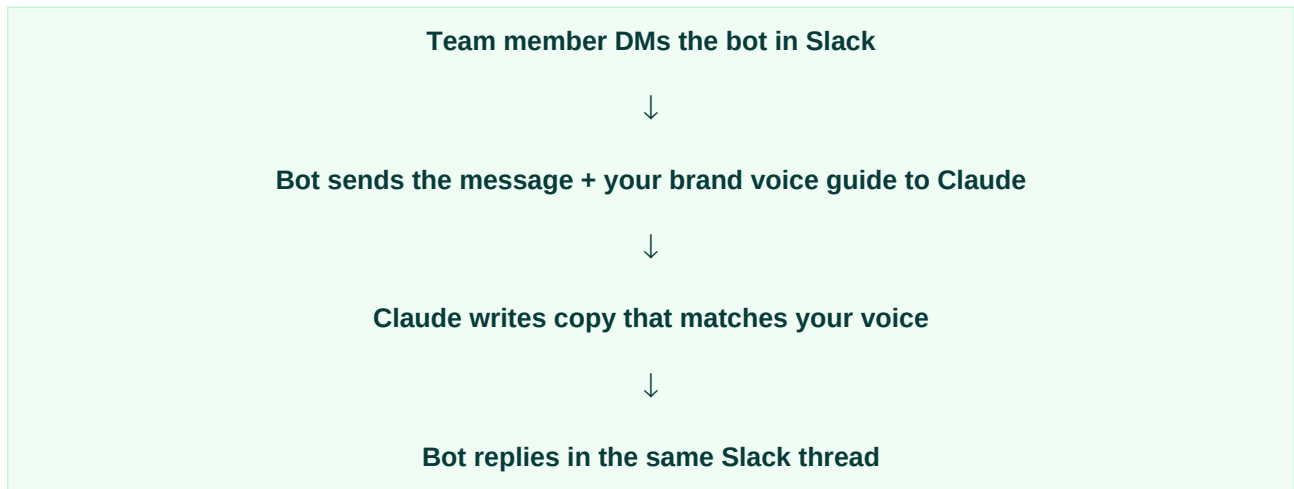
Before you start, collect these:

- A Slack workspace where you can create apps (admin permission required).
- An Anthropic API key (console.anthropic.com).
- A GitHub account (free).
- A Railway account (free to start, around \$5 a month once running).
- Your brand voice guidelines as a document (voice, tone, words to avoid, sample copy, platform rules).

Total setup time: about 2 hours the first time, including writing the system prompt. Every hour after that is pure time saved across your organization.

How It Works

The architecture is simple:



The bot runs as a small Node.js app on Railway. Railway pulls the code from a private GitHub repo. When you update the code on GitHub, Railway redeploys automatically. You never touch a server.

Thread memory is handled by Slack itself. When a team member replies "make it shorter" in the same thread, the bot pulls the full thread history and sends it back to Claude. No database required.

Step 1: Write Your System Prompt (30 minutes)

This is the most important step. The system prompt is the document Claude reads every time someone messages the bot. It defines your brand voice, the rules, and how the bot should behave.

A strong system prompt includes:

- **Voice and tone.** How does your brand sound? Direct? Warm? Formal? Give specific adjectives and examples.
- **Words and phrases to avoid.** List AI tells ("delve," "tapestry," "it's important to note"). List org-specific banned phrases.
- **Platform-specific rules.** Character limits, CTA placement, structure for email vs. X vs. LinkedIn vs. Instagram vs. long-form blog posts.
- **Calibration quotes.** 5 to 10 sentences from your best past copy. These teach Claude the rhythm of your voice better than any rule.
- **Request handling rules.** Tell the bot whether to ask clarifying questions (usually no, to save API costs) or to write immediately with best-guess assumptions.

Save this as a single document. You will paste it into the bot's code as a constant.

Step 2: Create the Slack App (15 minutes)

1. Go to api.slack.com/apps and click Create New App → From scratch.
2. Name it (e.g., "Brand Copywriting Bot") and pick your workspace.

3. Under **OAuth & Permissions**, add these Bot Token Scopes: im:history, im:write, im:read, chat:write, reactions:write.
4. Under **Socket Mode**, toggle it ON. Create an App-Level Token with the connections:write scope. Save this token (starts with xapp-).
5. Under **Event Subscriptions**, enable events and subscribe to message.im.
6. Under **Install App**, install to your workspace. Save the Bot User OAuth Token (starts with xoxb-).

Socket Mode matters: it lets the bot receive messages without exposing a public URL. That is why you do not need a server or DNS setup.

Step 3: Build the Bot (45 minutes)

The entire app is about 100 lines of JavaScript in a single file. The core loop:

1. Listen for DM messages.
2. Add an hourglass reaction so the sender knows the bot is thinking.
3. Build the conversation history from the Slack thread.
4. Send it to Claude with your system prompt.
5. Reply in the same thread.
6. Remove the hourglass reaction.

You do not need to write this from scratch. You can ask Claude Code (or ChatGPT or any coding assistant) to build it for you. Tell it:

"Build a Slack bot in Node.js using @slack/bolt in Socket Mode and @anthropic-ai/sdk. It should respond to DMs only, use thread history for context, and use my system prompt file."

You will end up with four files:

- **index.js**: the main app.
- **system-prompt.js**: your brand voice document, exported as a string.
- **package.json**: Node dependencies (@slack/bolt, @anthropic-ai/sdk).
- **.env.example**: a template listing the environment variables (the tokens you created above).

Step 4: Push to a Private GitHub Repo (10 minutes)

1. Create a new private repo on GitHub (name it whatever you want).
2. From your terminal, inside the bot folder, run the commands below.

```
git init
git add .
```

```
git commit -m "Initial bot"
git remote add origin https://github.com/YOUR-USERNAME/YOUR-REPO.git
git push -u origin main
```

Keep the repo private. It does not contain your API keys (those live in environment variables), but it does contain your system prompt, which is your brand IP.

Step 5: Deploy to Railway (10 minutes)

1. Go to **railway.app** and sign in with your GitHub account.
2. Click New Project → Deploy from GitHub Repo.
3. Select the repo you just created.
4. Once the project loads, click the Variables tab and add your environment variables (listed below).
5. Railway auto-deploys the moment you add the variables.

Variable	Value
SLACK_BOT_TOKEN	your xoxb- token
SLACK_APP_TOKEN	your xapp- token
ANTHROPIC_API_KEY	your Claude API key

Within 60 seconds, your bot is live.

Step 6: Test It

DM the bot in Slack. Try something specific:

"Write a launch email for our new product. Audience is existing customers. Tone should be warm and direct, with one clear call to action."

You should see an hourglass reaction, then a reply in your voice, in the right format for email, with a single CTA. If the output feels off, the problem is almost always in the system prompt. Edit it, push to GitHub, and Railway redeploys automatically.

Cost Projections

Claude API costs scale with usage. Here is what to budget for an organization of 50 people using the bot moderately:

Usage	Exchanges / person / day	Sonnet 4.6	Opus 4.6
Light	2	~\$75	~\$380
Medium	4	~\$240	~\$1,200
Heavy	6	~\$480	~\$2,400

Plus around \$5/month for Railway hosting.

Claude Sonnet 4.6 handles brand-voice copywriting well. Start there. Reserve Opus for high-stakes pieces by setting the `CLAUDE_MODEL` environment variable per project or per message.

Two more ways to reduce cost:

- **Tell the bot never to ask clarifying questions.** One clarifying question doubles the cost of a simple request because it turns a one-shot into a four-message thread. Instruct the bot to make best-guess assumptions and note them at the end.
- **Enable prompt caching** (an Anthropic feature) to avoid paying full price for resending your system prompt on every message. Caching cuts that cost by about 90 percent.

What To Add Later

Once the bot is running, you can extend it:

- **Past content library.** Upload past emails, blog posts, and landing pages to a folder. Modify the bot to retrieve the most relevant past piece and include it as context in the Claude call. This dramatically improves voice match for long-form copy.
- **Multiple voices.** If your organization has multiple spokespeople or brands, load different system prompts based on the Slack channel or user.
- **Logging.** Store every request and response in a simple Google Sheet or Airtable so you can review output quality over time and spot patterns you want to improve.

Maintenance

After the initial build, maintenance is simple:

- **Update the system prompt** whenever you refine your brand voice or launch a new campaign. Push to GitHub. Railway redeploys in a minute.
- **Rotate the Anthropic API key** quarterly for security. Update the Railway variable.
- **Watch your Anthropic billing dashboard** the first month to confirm usage matches your projection.

Your marketing team reclaims hours every week. Your staff gets on-brand copy on demand. The next version of your organization does more with the same team, because the bottleneck is no longer a person. It is a tool you own.

Want help building this for your team?

Seedscale builds custom Slackbots, AI workflows, and digital systems that let small teams ship like big ones.

Reach out at **seedscale.agency**.